AICL

Architecture Compilation Language

Complete User Manual

Version 1.0.0

If the compiler cannot explain why it generated a line,
it should not generate it.

Philippe-Antoine

June 2026

Table of Contents

1. Introduction

AICL (Architecture Compilation Language) is a specification-first programming language where software is defined through structured architectural intent rather than implementation instructions. Instead of writing executable code directly, developers specify objectives, architectural layers, anticipated failure modes, recovery procedures, constraints, validation criteria, and optimization goals. The AICL compiler then transforms these high-level architectural specifications into executable software through a multi-stage compilation pipeline that produces not only code but also a cryptographic Proof of Origin for every generated artifact.

The fundamental thesis of AICL is: **a compiler that cannot explain why it generated a line should not generate it.** This principle of explicable compilation means that every line of generated code must have a traceable provenance chain linking it back to the original specification. Furthermore, this property must be independently verifiable without trusting the compiler itself, solving the "Trust me bro" problem that plagues both traditional compilers and AI code generators.

Core Properties

AICL is built on five key properties that distinguish it from all other programming languages and code generation systems. These properties work together to create a compilation system where trust is mathematically justified rather than socially assumed:

Mandatory Risk/Recovery - Every Risk must have a Recovery. Error handling is not optional; it is a structural requirement of the language. You cannot write an AICL program without specifying what happens when things go wrong.

Validation to Tests - Every Validation section generates a test. Test coverage is not something you add later; it is a structural property of the compilation. Coverage is measured as |{A in Artifacts : A has provenance chain}| / |Artifacts|, and the target is always 1.0 (100%).

Explainable Artifacts - Every generated line of code has a traceable provenance chain. The compiler records why it made each decision, what specification element triggered it, and what pattern it used. This is not logging; it is a formal record of compilation reasoning.

Measurable Audit Coverage - Audit coverage equals auditable artifacts divided by total artifacts. The target is 100%, and this is a verifiable property of the compilation artifact, not just a feature of the compiler.

Independently Verifiable Proofs - Proof of Origin files can be verified without the compiler. The independent verifier is approximately 200 lines of Python using only the standard library, with zero AICL dependencies. It performs 8 verification checks and produces a deterministic VALID or INVALID verdict.

2. Installation

From PyPI

```
pip install aicl
```

From Source

```
git clone https://github.com/AFKmoney/AICL.git
cd AICL
pip install -e .
```

Requirements

AICL requires Python 3.10 or later. No external dependencies are required for core compilation. The independent verifier uses only Python standard library modules (json, hashlib, sys, os). The self-healing runtime, ownership model, and cryptographic signing features are all included in the base installation.

Verify Installation

```
aicl version
# Output: AICL v1.0.0
```

3. Quick Start: Your First Program

Let us create a minimal AICL program from scratch and compile it. This example will demonstrate the core workflow: write a specification, compile it, verify the proof, and explore the generated artifacts.

Step 1: Write the Specification

Create a file called **hello.aicl** with the following content. This is a complete, valid AICL program at Level 1 (Architecture). It specifies a goal, an architectural layer, a risk with its recovery, and a validation criterion:

```
Goal: Greet the user

Layer: Greeting Engine

Risk: User name is empty
Recovery: Use default name "World"

Validation: Greeting is produced for the user
```

Step 2: Compile

```
aicl compile hello.aicl --output-dir output
```

The compiler produces four files in the output directory: the main application (main.py), the test suite (test_main.py), the architecture tree (architecture_tree.txt), and the Proof of Origin (main.aicl-proof). The output will look like:

```
Compilation successful
Output directory: output
Main file: output/main.py
Test file: output/test_main.py
Architecture tree: output/architecture_tree.txt
Proof of Origin: output/main.aicl-proof
TODOs remaining: 0
Fully compiled: Yes
Audit coverage: 100.0%
Proof of Origin: VALID
```

Step 3: Verify the Proof Independently

The Proof of Origin can be verified without using AICL at all. The independent verifier uses only Python standard library:

```
python tools/verify_proof.py output/main.aicl-proof --verbose
```

This produces a detailed report of 8 verification checks, each showing PASS or FAIL, and a final verdict of VALID or INVALID. No AICL compiler, parser, or project dependency is used.

Step 4: Explore the Artifacts

```
# View the architecture tree
aicl tree hello.aicl

# Explain why each line was generated
aicl explain --proof output/main.aicl-proof

# Audit compilation coverage
aicl audit --proof output/main.aicl-proof

# Verify proof integrity
aicl proof output/main.aicl-proof --verify
```

4. Language Structure

AICL has 10 progressive levels, each adding a new architectural dimension. Level 1 (Architecture) is mandatory for all programs. Levels 2 through 10 are optional and can be combined freely based on the needs of your specification. The levels are designed so that each one introduces a conceptually distinct concern, preventing the conflation of architecture, data, behavior, and security that plagues general-purpose languages.

Level	Name	Keywords	Purpose
1	Architecture	Goal, Layer, Validation, Risk, Recovery, Constraint, Sublayer	System structure and risk management
2	Entities	Entity, field: type	Data structures

Level	Name	Keywords	Purpose
3	Behaviors	Behavior, Input, Output, Action	What entities do
4	Conditions	When, Then	Declarative conditional rules
5	Events	On, Action	Event-driven architecture
6	Concurrency	Parallel	Concurrent layer execution
7	Optimization	Optimize, Priority	Performance targets
8	Learning	Learn, Adapt, Based	Adaptive behavior
9	Security	Security, Encrypt, Protect	Security requirements
10	Native Code	Native	Inline code in other languages

4.1 Level 1: Architecture (Mandatory)

Level 1 is the foundation of every AICL program. It defines the system's purpose, its architectural layers, the risks it faces, how it recovers from those risks, and how its success is validated. These elements are not optional documentation; they are mandatory language constructs that directly influence the generated code. Every Goal produces an application class, every Layer produces an initialization method, every Risk/Recovery pair produces error handling code, and every Validation produces a test method.

Goal

The Goal defines the system's single overarching objective. It becomes the main class name and the docstring of the generated application. A program must have at least one Goal. The Goal should be a concise, declarative statement of what the system accomplishes:

```
Goal: Build a real-time chat application
```

Layer and Sublayer

Layers define the architectural decomposition of the system. Each Layer becomes a major component in the generated application, with its own initialization method and state management. Sublayers provide further decomposition within a Layer. The compiler uses the Layer hierarchy to determine initialization order, dependency relationships, and code organization:

```
Layer: Networking
Sublayer: WebSocket client
Sublayer: Message serialization
Sublayer: Heartbeat system

Layer: Chat Logic
Sublayer: Message routing
Sublayer: User management
```

Risk and Recovery

Risk/Recovery pairs are the heart of AICL's approach to error handling. Every Risk must have a corresponding Recovery. This is not a suggestion; it is a structural requirement enforced by the compiler. The generated code includes a risk handler that maps runtime errors to their defined recoveries. The compiler also checks that Risk and Recovery sections are balanced and issues warnings if they are not:

```
Risk: Server becomes unavailable
Recovery: Reconnect automatically with exponential backoff

Risk: Network latency exceeds threshold
Recovery: Enable offline mode with message queue
```

Constraint

Constraints define system limitations and requirements that are not directly testable but must be respected by the generated code. They become constants and comments in the output, serving as architectural reminders. Constraints can specify performance requirements, resource limits, or compatibility requirements:

```
Constraint: Must support at least 100 concurrent users
Constraint: Maximum memory usage 512 MB
Constraint: Messages must be delivered in order
```

Validation

Validations define success criteria for the system. Each Validation section generates a test method in the test suite. Validations should be specific and measurable when possible. The compiler can detect when a Validation uses vague language (like "works correctly") and issue a warning suggesting more precise wording. Well-written Validations lead to meaningful, testable code:

```
Validation: Send and receive messages successfully
Validation: Average latency below 100 ms
Validation: No data loss after reconnect
```

4.2 Level 2: Entities

Entities define the data structures of your system. Each Entity becomes a dataclass (in Python) or a struct (in Rust/Go) or a class (in JavaScript). Entities are the nouns of your specification: they represent the things your system manipulates. Fields have explicit types from AICL's type system, and the compiler generates type hints, constructors, and serialization methods automatically. Entities belong to the global scope and can be referenced by Behaviors, Conditions, and Events at any level:

```
Entity User
name: string
id: integer
status: string
avatar_url: string

Entity Message
content: string
timestamp: datetime
```

```
sender: User

Entity Channel
name: string
id: integer
members: list
```

4.3 Level 3: Behaviors

Behaviors define what entities do. Each Behavior has a name, optional Inputs, optional Outputs, and an Action description. The compiler maps Action descriptions to concrete code through 30+ deterministic patterns. When no pattern matches, the sub-language parser handles explicit action specifications. This means that every Behavior compiles to real, executable code with zero TODOs. The provenance chain records which pattern was selected and why, making the compilation decision fully explainable:

```
Behavior SendMessage
Input: User Message
Output: DeliveredMessage
Action: transmit message to server and broadcast to channel

Behavior MovePaddle
Input: Player direction string
Action: Update paddle position based on direction
```

For precise control over behavior implementation, you can use the sub-language parser with explicit assignment and operation syntax:

```
Behavior MovePaddle
Input: Player direction string
Action: assign paddle_position += direction * speed
clamp paddle_position between 0 and screen_width
```

4.4 Level 4: Conditions

Conditions replace traditional if/else statements with declarative When/Then rules. Instead of imperative branching logic, you specify what should happen when a condition is true. The compiler generates the appropriate conditional code with proper state checks. Conditions make the specification's branching logic explicit and auditable, rather than burying it in nested if statements:

```
Condition:
When server unavailable
Then enable offline mode

Condition:
When network restored
Then sync offline messages

Condition:
When ball hits paddle
Then reflect ball velocity
```

4.5 Level 5: Events

Events define event-driven behavior using On/Action pairs. When an event occurs, the specified action is triggered. The compiler generates event listeners, callback handlers, and message dispatch code. Events are particularly useful for user interface interactions, network message handling, and system notification processing. Each Event becomes a registered handler in the generated application's event loop:

```
Event:
On message received
Action: display message in channel

Event:
On user join
Action: update user list

Event:
On collision
Action: apply physics reflection
```

4.6 Level 6: Concurrency

The Parallel keyword specifies that certain layers should execute concurrently. The compiler generates the appropriate concurrency mechanism for the target language (threading in Python, async tasks in Rust, promises in JavaScript, goroutines in Go). You specify which layers run in parallel, and the compiler decides the threading strategy. This declarative approach to concurrency avoids the complexity of manual thread management while ensuring that the architectural intent is preserved:

```
Parallel:
Rendering System
Input System
Physics Engine
```

4.7 Level 7: Optimization

Optimization directives tell the compiler which performance characteristics matter. The compiler generates monitoring code and optimization hints based on these directives. When combined with the auto-optimizer (aicl optimize), these directives guide architectural improvements such as layer consolidation, entity extraction, and behavior inlining. Optimization targets become measurable metrics in the generated application:

```
Optimize: Memory usage
Optimize: Network latency
```

4.8 Level 8: Learning

Learning and Adapt directives specify adaptive behavior. The Learn keyword defines what the system should learn, and the Adapt keyword defines how it should adjust based on what it learned. The compiler generates monitoring and adaptation scaffolding code. This level bridges the gap between static specification and

dynamic runtime behavior, allowing the system to improve over time based on observed patterns:

```
Learn: User behavior
Goal: improve message suggestions

Adapt: Graphics quality
Based on: device performance
```

4.9 Level 9: Security

Security directives specify encryption and protection requirements. The Encrypt keyword indicates what data should be encrypted, and the Protect keyword indicates what resources need access control. The compiler generates encryption wrappers and access check methods. These directives ensure that security is a first-class architectural concern, not an afterthought bolted on post-development:

```
Security:
Encrypt: message content
Protect: user credentials
```

4.10 Level 10: Native Code

The Native keyword provides an escape hatch for low-level implementation that cannot be expressed in AICL's declarative syntax. You can embed code in any language directly within the specification. The compiler passes native code blocks through to the target language output without modification. Use this level sparingly; every Native block reduces the auditability of the generated code because its provenance is external rather than derived from the AICL specification:

```
Native: python {
import numpy as np
matrix = np.array([[1, 2], [3, 4]])
result = np.linalg.inv(matrix)
}
```

5. Type System

AICL provides 11 built-in types that cover the vast majority of data modeling needs. These types map to appropriate types in each target language. The type system is intentionally simple: complex type operations are better expressed through Behaviors and Conditions. Each type has a deterministic mapping to the target language, ensuring that the same Entity definition produces idiomatic code in Python, Rust, JavaScript, and Go:

AICL Type	Python	Rust	JavaScript	Go	Description
string	str	String	string	string	Text data
integer	int	i32	number	int	Whole numbers

AICL Type	Python	Rust	JavaScript	Go	Description
float	float	f64	number	float64	Floating-point
boolean	bool	bool	boolean	bool	True/False
datetime	datetime	chrono::DateTime	Date	time.Time	Date and time
list	List[Any]	Vec	Array	[]T	Ordered collection
dict	Dict[str, Any]	HashMap	Object	map[string]T	Key-value mapping
set	set	HashSet	Set	map[T]struct{}	Unique collection
any	Any	Box	any	interface{}	Dynamic type
void	None	()	void	struct{}	No return value
bytes	bytes	Vec	Uint8Array	[]byte	Binary data

6. Compilation

AICL compilation is a 9-stage pipeline that transforms a specification into executable code with full provenance tracking. Each stage records its decisions in the provenance chain, producing a complete audit trail. The compilation is deterministic: the same input always produces the same output. This is a formal contract of the compiler, not just an implementation detail. Deterministic compilation is essential for proof verification, because it means the proof is reproducible and the hash bindings are stable across compilations.

6.1 Compilation Pipeline

Stage	Name	Description
1	Specification Parsing	Parse AICL source into AST (Abstract Syntax Tree)
2	Architecture Validation	Check structural completeness (Goal, Layer, Validation present)
3	Dependency Analysis	Build layer dependency graph, determine initialization order
4	Risk Analysis	Pair every Risk with a Recovery, warn about unpaired risks
5	Recovery Synthesis	Generate recovery logic + record provenance for each recovery
6	Code Generation	Generate target language source with 30+ deterministic patterns
7	Test Generation	Generate test suite from Validations + link to provenance

Stage	Name	Description
8	Optimization	Apply Optimize/Priority hints and architecture improvements
9	Final Construction	Produce final output files + Proof of Origin + cryptographic signature

6.2 Deterministic Behavior Patterns

The Behavior Pattern Library maps action descriptions to concrete code through 30+ deterministic patterns organized into 9 categories. When a Behavior's Action description matches a pattern, the compiler uses that pattern to generate idiomatic code. When no pattern matches, the sub-language parser handles explicit action specifications. This ensures zero TODOs in compiled output. Every compilation decision is recorded in the provenance chain, making the pattern selection process fully explainable and auditable:

Category	Patterns	Example Mapping
Movement	MOVE, MOVE_BALL, REFLECT_VELOCITY, CLAMP_POSITION	"Update paddle position" becomes MOVE pattern
Creation	CREATE, INIT_LAYER, CREATE_WINDOW	"Create new game" becomes CREATE pattern
Communication	BROADCAST, SEND_MESSAGE	"Transmit message" becomes BROADCAST pattern
Update	UPDATE, INCREMENT_SCORE, UPDATE_STATE	"Clamp ball position" becomes CLAMP pattern
Display	DISPLAY, RENDER_FRAME, HIGHLIGHT	"Draw game frame" becomes RENDER pattern
Validation	VALIDATE, CHECK_COLLISION	"Validate move" becomes VALIDATE pattern
Networking	CONNECT, RECONNECT, DISCONNECT	"Connect to server" becomes CONNECT pattern
Persistence	STORE, LOAD	"Save game state" becomes STORE pattern
Security	ENCRYPT_DATA, PROTECT_ACCESS	"Encrypt message" becomes ENCRYPT pattern

6.3 Multi-Language Targets

AICL v1.0 supports four target languages. Each target generator produces idiomatic code with proper error handling from Risk/Recovery pairs, entity structures as language-appropriate types, behavior methods with deterministic patterns, test suites from Validation sections, and project configuration files. The same specification compiles to fundamentally different but architecturally equivalent programs in each target

language:

Target	Status	Entity Mapping	Error Handling	Test Framework	Config File
Python	Mature (default)	dataclass	try/except	pytest	pyproject.toml
Rust	Beta	struct	Result	#[test]	Cargo.toml
JavaScript	Beta	ES6 class	try/catch + async/await	Jest	package.json
Go	Beta	struct	error type	testing package	go.mod

```
# Compile to different targets
aicl compile pong.aicl --target rust --output-dir rust_output
aicl compile pong.aicl --target javascript --output-dir js_output
aicl compile pong.aicl --target go --output-dir go_output
```

7. Proof of Origin

The Proof of Origin is the central artifact of AICL compilation. It is a self-contained, cryptographically bound file that proves every generated artifact has traceable provenance. In AICL's architecture, the Proof is not a side product; it is the primary product. Code, explanation, and audit are views on the proof. This architectural shift means that the proof contains everything needed to verify the compilation without the compiler itself, and it can be independently verified by any third party using only the Python standard library.

7.1 Proof Format v2.0

Proof files are self-contained JSON with the .aicl-proof extension. Format v2.0 embeds the source text, generated source code, and generated tests directly in the proof, making it fully self-contained. This means that verification can be performed with only the proof file, without needing access to the original source or generated code. The SHA-256 hash bindings ensure that any tampering with the embedded code would be detected immediately:

```
{
"format_version": "2.0",
"compiler_version": "1.0.0",
"timestamp": "2026-06-13T12:00:00Z",
"source_hash": "sha256:...",
"program_hash": "sha256:...",
"test_hash": "sha256:...",
"source_text": "Goal ...",
"generated_source": "class Application ...",
"generated_tests": "def test_ ...",
"records": [...],
"artifacts": [...],
"formal_properties": {...},
```

```
"audit_coverage": {...},
"explicability_coverage": {...}
}
```

7.2 Independent Verification

The independent verifier (tools/verify_proof.py) is approximately 200 lines of Python that uses only the standard library (json, hashlib, sys, os). It has zero AICL dependencies. This is critical: it means that the verification of AICL's output does not depend on AICL itself. The verifier can be audited by anyone with basic Python knowledge, and its simplicity makes it easy to verify that the verifier itself is correct. This solves the "Trust me bro" problem where AICL says AICL is correct; now an independent program says the proof is valid:

```
# Verify a proof
python tools/verify_proof.py output/main.aicl-proof

# Verbose output showing all checks
python tools/verify_proof.py output/main.aicl-proof --verbose
```

7.3 The 8 Verification Checks

The independent verifier performs 8 checks, each verifying a specific property of the proof. All 8 checks must pass for the proof to be considered VALID. A single failure results in an INVALID verdict. These checks verify that the proof is well-formed, that the hash bindings are correct, that every artifact has provenance, and that the formal properties hold:

#	Check	What It Verifies
1	format_version	Proof format version is supported (1.0 or 2.0)
2	source_hash_binding	SHA-256(source_text) matches the declared source_hash
3	program_hash_binding	SHA-256(generated_source) matches the declared program_hash
4	test_hash_binding	SHA-256(generated_tests) matches the declared test_hash
5	no_orphan_artifact_property	Every generated artifact has at least one provenance chain
6	complete_coverage_property	Audit coverage = |auditable| / |total| = 1.0 (100%)
7	record_artifact_linkage	All provenance records reference valid artifact indices
8	artifact_consistency	Orphan status is consistent with provenance linkage

8. Audit System

The audit system verifies that every generated artifact can be traced to its originating specification through a complete provenance chain. Audit coverage is defined as the ratio of auditable artifacts to total artifacts, and the target is always 1.0 (100%). The audit system is not a separate tool; it is an integral part of the compilation pipeline that runs automatically and produces its results in the Proof of Origin. The audit can also be performed after compilation by reading the proof file, without needing the compiler.

Three Formal Properties

The audit system is built on three formal properties that are verifiable as properties of the compilation artifact, not just features of the compiler. This distinction is crucial: it means that anyone can verify these properties using the independent verifier, without trusting AICL at all:

The No-Orphan Property - Every generated artifact has at least one provenance record. An orphan artifact is one that exists in the generated code but has no explanation for why it was generated. The No-Orphan Property guarantees that this never happens. It is verified by check #5 of the independent verifier.

The Complete Coverage Property - Audit coverage equals 1.0. This means that every single artifact in the generated code has a traceable provenance chain. Coverage less than 1.0 means some artifacts were generated without explanation. It is verified by check #6 of the independent verifier.

The Hash Binding Property - SHA-256 hashes bind the proof to the generated code. Any modification to the source text, generated code, or tests would change the hash and cause verification to fail. It is verified by checks #2, #3, and #4 of the independent verifier.

```
# Run audit from a proof file
aicl audit --proof output/main.aicl-proof
```

9. CLI Reference

AICL provides a comprehensive command-line interface with 10 commands covering the full development workflow from specification to verification. Each command has a specific purpose in the development pipeline, and they can be combined to create powerful workflows. The CLI is installed as the **aicl** command when you install the AICL package.

Command	Description	Example
aicl compile	Compile AICL source to target language	aicl compile app.aicl --output-dir out
aicl parse	Parse and display AST	aicl parse app.aicl
aicl tree	Display architecture tree	aicl tree app.aicl
aicl check	Check for errors and warnings	aicl check app.aicl
aicl explain	Explain compilation provenance	aicl explain --proof out/main.aicl-proof

Command	Description	Example
aicl audit	Audit compilation coverage	aicl audit --proof out/main.aicl-proof
aicl proof	Inspect and verify Proof of Origin	aicl proof out/main.aicl-proof --verify
aicl verify	Verify specification quality	aicl verify app.aicl
aicl optimize	Optimize architecture	aicl optimize app.aicl
aicl version	Display version	aicl version

Compile Options

Option	Description	Default
--output-dir DIR	Output directory for generated files	output
--target LANG	Target language (python, rust, javascript, go)	python
--verbose	Show detailed compilation output	off

Verify Options

Option	Description
--completeness	Check only completeness (all required elements present)
--coherence	Check only coherence (no contradictions or dangling references)
--satisfaction	Check only satisfaction (specification is implementable and testable)

10. Specification Verification

The aicl verify command checks specifications at three levels before compilation. This catches specification errors early, before they produce incorrect code. Specification verification is separate from compilation verification (which checks the proof). Specification verification checks the quality of the input; proof verification checks the quality of the output. Both are needed for a complete quality assurance pipeline.

Completeness - Verifies that all required elements are present. A valid AICL program must have at least one Goal, one Layer, and one Validation. Entities and Behaviors are recommended for non-trivial programs but not strictly required. Risk/Recovery pairing is checked: every Risk should have a Recovery.

Coherence - Checks for contradictions and dangling references. No duplicate entity names, no duplicate behavior names, no cyclic layer hierarchies, and all constraints must have non-empty

descriptions.

Satisfaction - Verifies that the specification is implementable and testable. Validations should use measurable language (must, should, equals, etc.). The architecture should be implementable with the given layers. Risk coverage should be sufficient (recoveries exist for all risks).

```
# Full verification (all three levels)
aicl verify pong.aicl

# Individual levels
aicl verify pong.aicl --completeness
aicl verify pong.aicl --coherence
aicl verify pong.aicl --satisfaction
```

11. Module System

AICL supports multi-file programs through an import and module mechanism. Modules allow you to split large specifications into manageable files, share common definitions across projects, and build reusable libraries of architectural patterns. Cross-file provenance ensures that even when specifications span multiple files, the provenance chain remains complete and traceable. Every imported element retains its origin information, so the audit system can track artifacts back to their source module:

```
# In file: common/entities.aicl
Entity User
name: string
id: integer

# In file: chat.aicl
Import: common/entities.aicl

Goal: Build a chat app
Layer: Chat Logic
```

12. Self-Healing Runtime

AICL's self-healing runtime extends compile-time provenance into execution. It provides three key capabilities that bridge the gap between specification and running system. The runtime monitors the conditions defined in your Risk sections, automatically executes Recovery actions when those risks materialize, and records every runtime event in a provenance chain that extends the compilation provenance into the execution domain. This means that not only can you trace why a line of code was generated, but you can also trace what happened when that line was executed:

Risk monitoring - Runtime checks detect when risks materialize. Each Risk from the specification becomes a monitored condition that is checked during execution.

Automatic recovery - Recovery actions execute with retry and backoff. When a risk is detected, the corresponding Recovery is automatically invoked, with configurable retry policies.

Runtime provenance - Every event is recorded in a provenance chain. This creates an execution audit trail that complements the compilation audit trail.

```
from aicl.runtime import RuntimeEnvironment

env = RuntimeEnvironment()
env.register_risk_recovery(
"network_failure",
risk_condition=lambda: not check_network(),
recovery_action=lambda: reconnect(),
)
result = env.run(application_main)
```

13. Ownership Model

AICL provides an ultra-simple ownership model derived from the Layer/Entity structure of your specification. Each Layer owns the entities defined within its scope, and ownership determines lifecycle management, access patterns, and memory responsibility. The ownership model is derived automatically from the specification, not manually specified. This means that the architectural structure you define directly determines the memory management strategy of the generated code, creating a tight coupling between design intent and implementation:

Layer ownership - Each Layer owns its entities. When a Layer is destroyed, its entities are destroyed with it. This prevents dangling references and use-after-free errors.

Ownership report - The compiler generates an ownership report showing which Layer owns which Entity, the lifecycle dependencies, and the access patterns.

Generated code - Ownership code is generated automatically, including initialization, cleanup, and access control methods based on the ownership hierarchy.

14. Architecture Optimization

The aicl optimize command performs autonomous architecture optimization based on the specification. It analyzes the specification for improvement opportunities and suggests specific, actionable changes. The optimizer runs multiple iterations and tracks improvement scores. Each suggestion includes a risk assessment and affected components. Optimization strategies include layer consolidation (merging layers that share patterns), entity extraction (finding common fields that could become base entities), behavior inlining (simplifying trivial behaviors), and validation strengthening (making vague validations more specific):

```
aicl optimize pong.aicl
```

The optimizer produces a detailed report with numbered actions, risk levels, affected components, and before/after descriptions. Each action is a concrete, implementable change that would improve the specification's architectural quality. The improvement score reflects how much the suggested changes would improve the specification's structure, not its functionality.

15. Cryptographic Proof Signing

Proof files can be cryptographically signed with the compiler's key, providing an additional layer of integrity verification beyond hash binding. Cryptographic signing ensures that the proof was produced by a specific compiler version and has not been tampered with since compilation. Cross-compilation proof chains link successive compilations, creating an auditable history that shows how the generated code evolved over time. This is particularly important in regulated environments where compilation provenance must be traceable to a specific tool and version:

```
from aicl.crypto_signing import create_signed_proof, verify_signed_proof

# Sign a proof
signed = create_signed_proof(proof_dict, key_seed="compiler-key")

# Verify the signature
result = verify_signed_proof(signed)
# result["signature_present"] == True
# result["proof_hash_valid"] == True
```

16. Complete Examples

This section presents two complete, working AICL programs that demonstrate different levels of the language. Each example can be compiled and verified independently. These are not toy examples; they demonstrate real architectural patterns and show how AICL scales from simple to complex specifications.

Example 1: Blue Square (Level 1)

This is a simple graphical application that uses only Level 1 (Architecture) features. It demonstrates the fundamental structure of an AICL program: Goal, Constraints, Risks with Recoveries, Layers, and Validations:

```
# AICL Example: Blue Square
Goal: Display a blue square on screen

Constraint: Must use hardware acceleration when available

Risk: Graphics device unavailable
Recovery: Use software renderer

Risk: Shader compilation failure
Recovery: Load fallback shader

Layer: Application Window
Layer: Renderer
Sublayer: Device creation
Sublayer: Resource management
Sublayer: Shader system
Layer: Blue Square
Layer: Blue Shader

Validation: Blue square is visible on screen
Validation: Application remains responsive for 60 seconds
```

Example 2: Pong Game (Levels 1-6)

This Pong game specification demonstrates Levels 1 through 6, including Entities, Behaviors, Conditions, Events, and Concurrency. It shows how the language levels combine to create a complete, auditable specification of a non-trivial system:

```
# AICL Example: Pong Game
Goal: Build a Pong game

Constraint: Must run at 60 FPS minimum

Risk: Ball leaves play area
Recovery: Clamp ball position to boundaries

Risk: Score not displayed correctly
Recovery: Reinitialize score display

Layer: Window Manager
Layer: Rendering System
Sublayer: Sprite rendering
Sublayer: Text rendering
Layer: Input System
Layer: Physics Engine
Sublayer: Ball movement
Sublayer: Collision detection
Layer: Game Logic
Sublayer: Score tracking
Sublayer: Game state management

Entity Player
name: string
score: integer
paddle_position: float

Entity Ball
x: float
y: float
velocity_x: float
velocity_y: float

Behavior MovePaddle
Input: Player direction string
Action: Update paddle position based on direction

Behavior UpdateBall
Input: Ball
Action: Update ball position and handle collisions

Condition:
When ball hits paddle
Then reflect ball velocity

Condition:
When score reaches 10
Then end match

Event:
On key press
Action: move corresponding paddle
```

```
Parallel:
Rendering System
Input System
Physics Engine

Validation: Complete one playable match
Validation: Maintain 60 FPS during gameplay
Validation: Score increments correctly on point
```

17. Troubleshooting

This section covers common issues and their solutions. Most problems fall into a few categories: specification errors, compilation warnings, verification failures, and target-specific issues.

Specification Errors

Error	Cause	Solution
Missing Goal	No Goal section in the program	Add a Goal: section at the top of the file
Missing Layer	No Layer section in the program	Add at least one Layer: section
Missing Validation	No Validation section	Add at least one Validation: section
Unpaired Risk	Risk without Recovery	Add a Recovery: section after the Risk
Duplicate Entity	Two entities with the same name	Rename one of the entities to be unique
Cyclic Layer Hierarchy	Layer depends on itself	Restructure layers to remove the cycle

Compilation Warnings

Warning	Cause	Solution
Validation not testable	Validation uses vague language	Use measurable terms: 'must equal', 'should be below'
Goal underspecified	No entities or behaviors defined	Add Entity and Behavior sections
Risk coverage insufficient	More risks than recoveries	Add Recovery sections for all Risks

Verification Failures

Failed Check	Meaning	Action
source_hash_binding	Source text was modified after compilation	Recompile from the original source
program_hash_binding	Generated code was modified	Recompile to regenerate the proof
no_orphan_artifact_property	An artifact has no provenance	Report as a compiler bug
complete_coverage_property	Audit coverage is less than 100%	Report as a compiler bug

18. Keyword Reference

AICL has 27 reserved keywords across 10 language levels. Each keyword has a specific purpose and syntax. Keywords are case-sensitive and must appear at the beginning of a line (except for sub-keywords like Sublayer, When, Then, On, Input, Output, Action, Encrypt, Protect, Based, and Priority which must be indented within their parent section). The following table provides a complete reference for all keywords:

Keyword	Level	Syntax	Purpose
Goal	1	Goal:	Define the system objective
Constraint	1	Constraint:	Define a system limitation
Risk	1	Risk:	Define a failure condition
Recovery	1	Recovery:	Define a corrective action
Layer	1	Layer:	Define an architectural layer
Sublayer	1	Sublayer:	Define a sub-layer within a Layer
Validation	1	Validation:	Define a success criterion
Entity	2	Entity	Define a data entity with fields
Behavior	3	Behavior	Define what entities do
Input	3	Input:	Define behavior input
Output	3	Output:	Define behavior output
Action	3/5	Action:	Define action to perform
Condition	4	Condition:	Define conditional behavior
When	4	When	Condition trigger

Keyword	Level	Syntax	Purpose
Then	4	Then	Condition consequence
Event	5	Event:	Define event-driven behavior
On	5	On	Event trigger
Parallel	6	Parallel:	Define concurrent execution
Optimize	7	Optimize:	Define optimization target
Priority	7	Priority:	Define optimization priority
Learn	8	Learn:	Define learning objective
Adapt	8	Adapt:	Define adaptive behavior
Based	8	Based on:	Adaptation criterion
Security	9	Security:	Define security requirements
Encrypt	9	Encrypt:	Encryption directive
Protect	9	Protect:	Protection directive
Native	10	Native: { code }	Inline code in another language

Minimal Valid Program

The smallest valid AICL program requires only three elements: a Goal, a Layer, and a Validation. Everything else is optional and adds more detail, more safety, and more auditable structure to the specification:

```
Goal: Hello World

Layer: Core

Validation: Output is produced
```

Comments

Lines starting with # are treated as comments and ignored by the parser. Comments can appear anywhere in the file and are useful for documenting the rationale behind architectural decisions, which is itself a form of provenance:

```
# This is a comment
Goal: Build a chat app # Inline comments are not supported
```